

OPERATING SYSTEMS

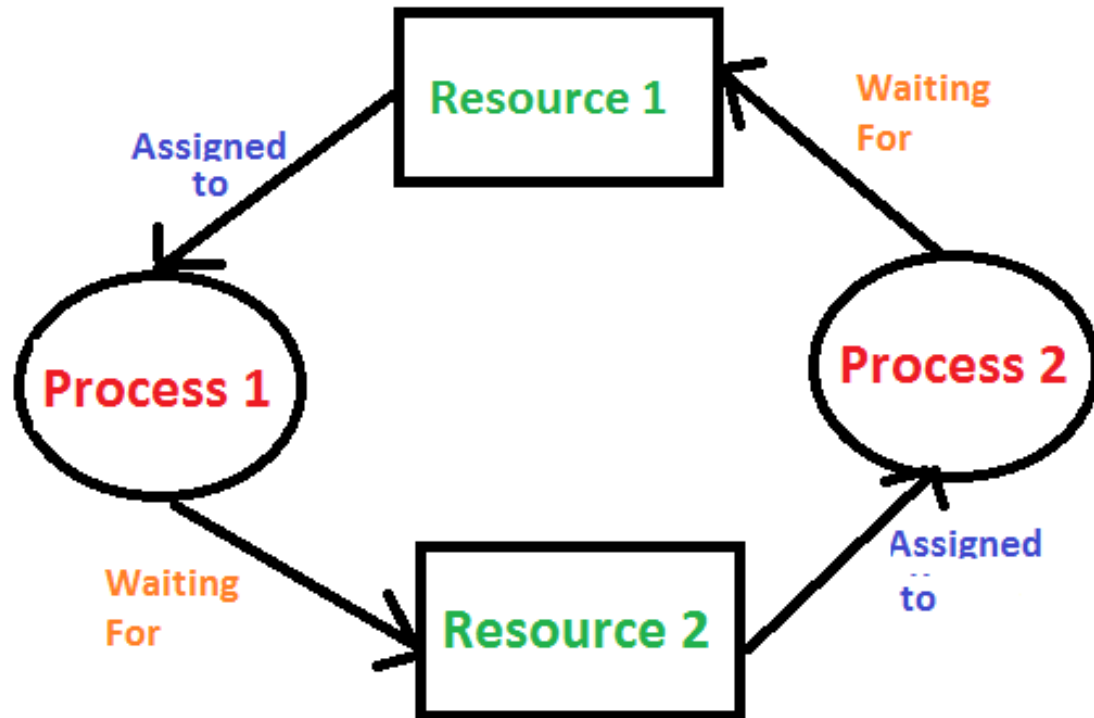
Process Coordination & Deadlocks

S.Thenmozhi

Department of Computer Applications

- System consists of finite number of resources which has to be shared among the competing process.
- Resource types $R_1, R_2, \dots, R_m$
 - *CPU cycles, files, memory space, I/O devices, etc*
- Each resource type R_i has W_i instances
 - Type: CPU ; 2 instances - CPU1, CPU2
 - Type: Printer ; 3 instances - printer1, printer2, printer3
- Each process utilizes a resource as follows:
 - **Request** resource
 - **Use** resource
 - **Release** resource

- Deadlock happens in operating system when two or more processes need some resource to complete their execution that is held by the other process
- Deadlock is a situation where a **set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.**

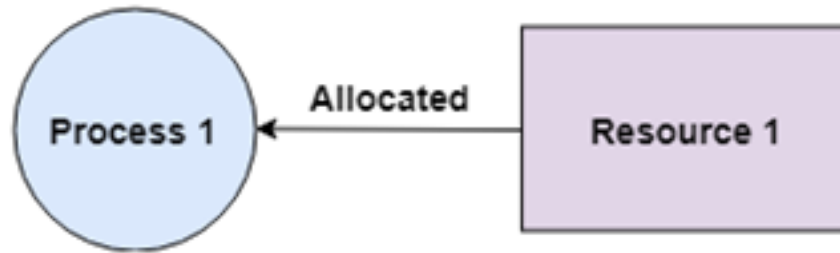


Necessary Conditions that causes deadlock

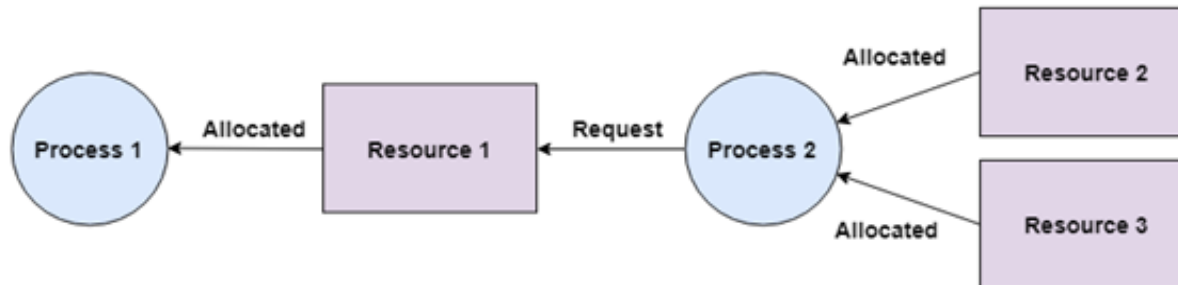
All 4 conditions holds simultaneously – Deadlock

1. Mutual exclusion
2. Hold and Wait
3. No preemption
4. Circular Wait

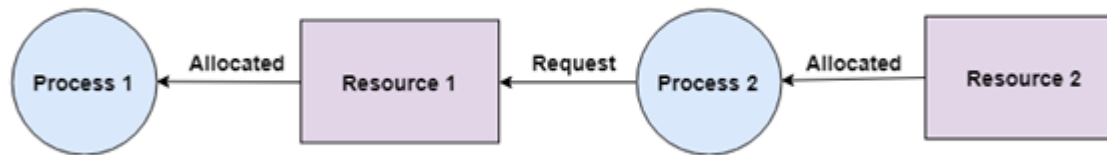
1. Mutual exclusion: only one process at a time can use a resource



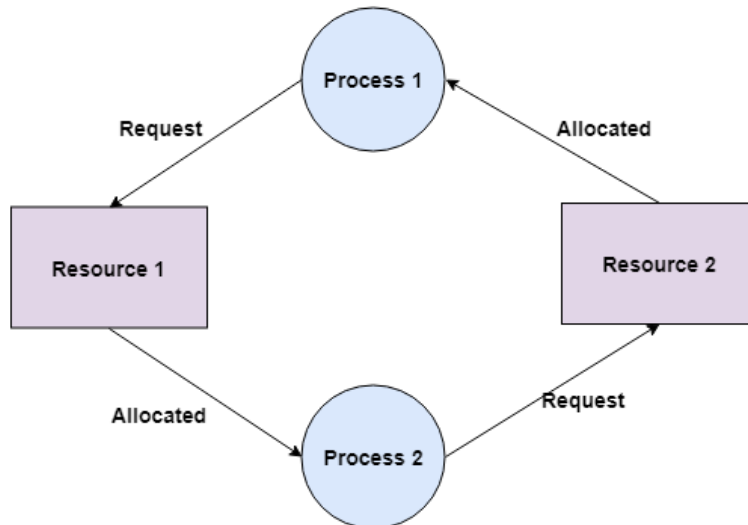
2. Hold and Wait: A process can hold multiple resources and still request more resources from other processes which are holding them.



3. No Preemption: A resource cannot be preempted from a process by force. A process can only release a resource voluntarily.

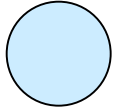


4. Circular Wait: A process is waiting for the resource held by the second process, which is waiting for the resource held by the third process and so on, till the last process is waiting for a resource held by the first process. This forms a circular chain

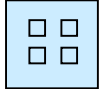


- **Resource-Allocation Graph**
 - Useful tool to describe and analyze deadlocks
 - Set of **vertices** V & Set of **edges** E
- **Vertex V - two types:**
 - $P = \{P_1, P_2, \dots, P_n\}$, the set of all the processes in the system
 - $R = \{R_1, R_2, \dots, R_m\}$, the set of all resource **types** in the system
- **Edges E – two types**
- **Request edge** – directed edge $P_i \rightarrow R_j$
 - Means P_i has requested (an instance of) R_j and now is waiting for it
- **Assignment edge** – directed edge $R_j \rightarrow P_i$
 - Means an instance of R_j has been assigned to P_i

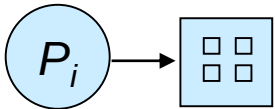
- Process



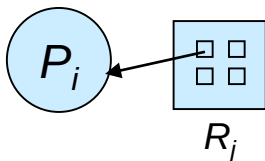
- Resource Type (with 4 dots for 4 instances in this specific example)



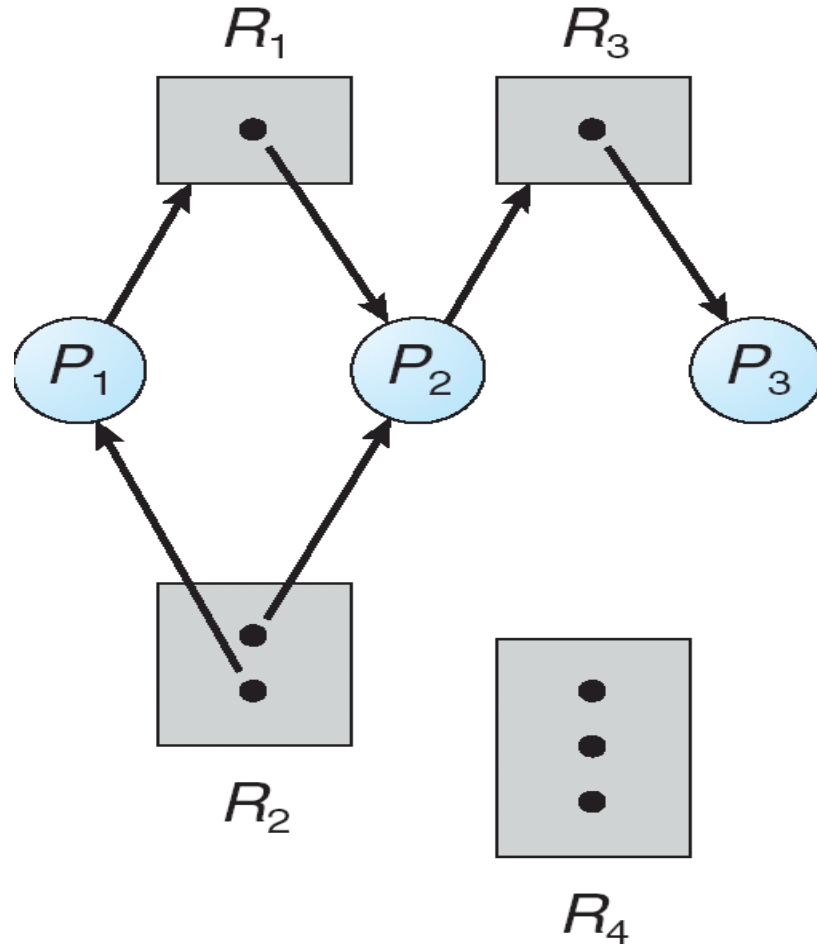
- *Request Edge* - P_i requests instance of R_j



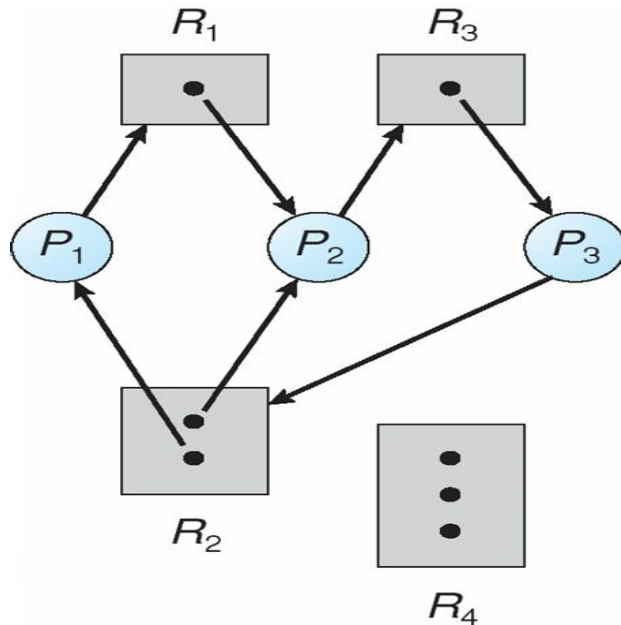
- *Assignment Edge* - P_i is holding an instance of R_j



Resource Allocation graph - Example



Resource Allocation graph –Deadlock State



If graph contains no cycles $\Rightarrow$ no deadlock

If graph contains a cycle $\Rightarrow$

- if only one instance per resource type, then deadlock
- if several instances per resource type, possibility of deadlock

Three ways to handle deadlocks:

1. Ensure that the system will *never* enter a deadlock state:
 - Deadlock prevention
 - Deadlock avoidance (Bankers algorithm)
2. Allow the system to enter a deadlock state and then *recover*
3. *Ignore* the problem and pretend that deadlocks never occur in the system (ostrich method)
 - Used by most operating systems, including UNIX
 - One reason: Handling deadlocks can be computationally expensive



THANK YOU

S. Thenmozhi

Department of Computer Applications

thenmozhis@pes.edu

+91 80 6666 3333 Extn 393